

COMPREHENSIVE PROJECT DOCUMENTATION & SUBMISSION REPORT

Course Module: Month 1: Python
Programming Mastery

Project Name: Personal Finance Manager

Author: Sagar mishra

Email:0173cs211056@gmail.com

Table of Contents

1. **Project Overview** (Goals, Objectives, and Scope)
 2. **Setup Instructions & Installation Guide** (Environment Setup and System Launch)
 3. **Code Structure & File Hierarchy** (Visual Directory Tree and Module Roles)
 4. **Detailed Code Explanation** (Module-by-Module Code Walkthrough)
 5. **Technical Details** (System Architecture, Data Structures, and Core Algorithms)
 6. **Comprehensive User Manual** (Functional Walkthrough of Options 1–7)
 7. **Visual Documentation Plan** (Directions for Required Screenshots)
 8. **Testing Evidence** (Automated Test Cases and CLI Validation Runs)
 9. **Troubleshooting Guide** (Common Execution Errors and Resolutions)
-

1. Project Overview

Project Description

The **Personal Finance Manager** is a terminal-based financial ledger program developed in Python. It provides users with a local environment to record daily transactions, monitor spending patterns across custom categories, search historical records, generate monthly analytical reports, and maintain file redundancy using automated backups [1].

Objectives

- **Data Persistence:** Achieve structured, local transaction persistence using flat-file CSV formats, avoiding external database software [1].
 - **Input Validation:** Enforce validation rules on all keyboard inputs to prevent corrupted or misaligned datatypes from entering the registry [1].
 - **Object-Oriented Design:** Use custom Python classes to model, format, and organize financial records [1].
 - **Modular Software Development:** Divide application logic into separate, cohesive modules (Input, Data Abstraction, Business Logic, and UI) to simplify maintenance [1].
-

2. Setup Instructions & Installation Guide

This step-by-step guide walks you through verifying your Python environment, configuring workspace files, and launching the application.

Step 1: System Requirements & Python Verification

The application uses core standard Python libraries and requires **Python 3.8 or higher** [1]. To check your current Python version, run:

```
codeBash
```

```
python --version
```

(On macOS/Linux systems, use `python3 --version`).

Step 2: Extracting or Cloning Project Files

Clone the repository or extract your project files to your desktop:

```
codeBash
```

```
cd "C:\Users\saura\OneDrive\Desktop\personal_finance_manager"
```

Step 3: Setting Up a Virtual Environment (Optional)

While the application uses standard libraries and doesn't require third-party dependencies, initializing a virtual environment keeps your workspace isolated:

```
codeBash
```

```
# Windows
```

```
python -m venv venv
```

```
.\venv\Scripts\activate
```

```
# macOS / Linux
```

```
python3 -m venv venv
```

```
source venv/bin/activate
```

Step 4: System Launch

Start the main application loop by executing main.py from the root directory:

```
codeBash
```

```
python main.py
```

3. Code Structure & File Hierarchy

The project files are arranged according to the following layout:

codeText

personal_finance_manager/ (Root Directory)

```
|
|— main.py          # Entry point for the application
|— README.md        # Quick-start documentation
|— requirements.txt  # Project dependencies list
|— .gitignore        # Prevents local user data from uploading to Git
|
|— data/             # Local transaction registry path (Auto-generated)
|   |— expenses.csv  # Active comma-separated database
|   |— backups/       # Automatically generated backups path
|
|— docs/             # Project manuals and guidelines path
|   |— project_documentation.md # This comprehensive documentation file
|
|— tests/            # Testing package
|   |— test_finance.py # Automated testing suite
|
|— src/              # Core source code package
|   |— __init__.py    # Identifies "src" as a Python package
|   |— expense.py     # Expense data model class
|   |— file_manager.py # CSV file operations & backup automation
|   |— reports.py     # Math aggregation, summary, and query functions
|   |— menu.py        # Interactive terminal UI and routing
|   |— utils.py       # Datatype validations and sanitization constants
```

4. Detailed Code Explanation

This section explains the logic and code structures used within each module.

4.1 Entry Point: main.py

This file is the startup script. It imports the user interface framework and launches the application's main loop:

```
codePython
```

```
from src.menu import run_app
```

```
if __name__ == "__main__":
```

```
    run_app()
```

- **Why it is structured this way:** By keeping main.py at the root, the system base path is set to the project root directory. This allows relative imports (like from src.utils import...) to resolve correctly when executing the program [1].

4.2 Data Model: src/expense.py

This module defines the Object-Oriented class used to represent financial transactions:

```
codePython
```

```
class Expense:
```

```
    def __init__(self, amount, category, date, description):
```

```
        self.amount = float(amount)
```

```
        self.category = category
```

```
        self.date = date
```

```
        self.description = description
```

```
    def __str__(self):
```

```
    return f"{self.date} | {self.category}: ₹{self.amount:.2f} -  
{self.description}"
```

```
def to_list(self):
```

```
    return [self.date, self.category, self.amount, self.description]
```

- **Role:**

- `__init__`: Standardizes raw values into their target types (e.g., converting amounts to floats).
- `__str__`: Implements formatting to simplify printing transactions to the CLI [1].
- `to_list()`: Formats instance properties into a list style suitable for writing directly as rows in the CSV file.

4.3 Data Storage: `src/file_manager.py`

Abstracts file read and write operations. It handles dynamic folder creation, data parsing, and backup automation [1].

- **Key Functions:**

- `ensure_data_dir()`: Checks if the target `/data/` folder exists on runtime. If missing, it creates the folder and initializes `expenses.csv` with the headers `['Date', 'Category', 'Amount', 'Description']` [1].
- `load_expenses()`: Parses the CSV file using `csv.DictReader` and returns a list of `Expense` objects. It is designed to handle errors, meaning if a single row is corrupted, it skips that row and continues loading the remaining transactions.
- `save_expenses(expenses)`: Iterates over the running list of `Expense` objects and writes them back to the active CSV database [1].
- `backup_data()`: Uses Python's `shutil` utility to copy the active CSV file to a timestamped file inside `data/backups/` [1].

4.4 Analytical Logic: `src/reports.py`

Performs computational calculations on sets of transactions without making disk or console calls.

- **Key Functions:**

- `generate_category_summary(expenses)`: Summarizes total spending across categories, returning a dictionary of mapped values.
- `generate_monthly_report(expenses, year_month)`: Filters active listings based on the provided year-month string (YYYY-MM). It calculates transaction counts, total expenditures, average spending per transaction, and category breakdowns.
- `search_expenses(expenses, query)`: Runs a substring match across categories and descriptions, returning a list of matching entries.

4.5 Keyboard Validation: `src/utils.py`

Contains validation checks to ensure only correctly formatted data enters the application [1].

- **Key Functions:**

- `validate_date(date_str)`: Uses `datetime.datetime.strptime` to verify if a date string matches the YYYY-MM-DD format.
- `validate_amount(amount_str)`: Verifies that numerical values are valid positive numbers.
- `validate_category(category_str)`: Normalizes casing and verifies if the input matches any value in the whitelisted array: ["Food", "Transport", "Entertainment", "Shopping", "Other"].

4.6 Terminal User Interface: `src/menu.py`

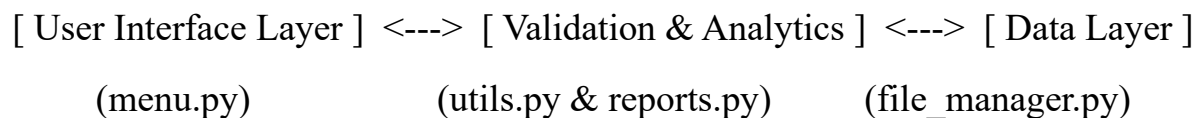
Coordinates the presentation layer. It displays CLI option listings, runs interactive command forms, captures keyboard entries, and routes data to other modules.

5. Technical Details

Architectural Design Pattern

The application uses a **layered architecture** to separate interface actions from computational logic and file I/O operations:

codeText



This separation of concerns makes the code easier to maintain and modify. For example, if you want to switch the interface from a text terminal to a graphic desktop application (such as Tkinter), you would only need to rewrite menu.py. The business logic (reports.py) and data storage modules (file_manager.py) would remain unchanged.

Data Structures

- **RAM Object Storage (List of Objects):** Transactions are stored in memory as a standard Python list containing Expense objects. This allows for fast lookup speeds and straightforward sorting and filtering.
- **Aggregations (Mapped Dictionary):** Categories are mapped to cumulative floats using standard key-value lookups:

codePython

Example memory structure returned by reports.py:

```
summary = {"Food": 1500.00, "Transport": 150.00, "Shopping": 2500.00}
```

Algorithms

- **Input Validation Loop:** When adding an expense, the application uses an loop to validate each input field individually. If a field fails validation, it displays a specific error message and prompts the user again without resetting the entire form:

codeText

```
[Start Add Form] -> [Prompt Amount] -> [Valid?] -No-> [Print Error & Re-Prompt]
```

|

Yes

v

[Prompt Category] ...

- **Robust CSV Parsing:** Rather than manually splitting lines by commas, the application uses Python's csv.DictReader. This avoids parsing bugs caused by commas in description fields (e.g., "Grocery, supermarket").

6. Comprehensive User Manual

Interactive Main Menu

When running python main.py, the terminal displays the active command prompt:

codeText

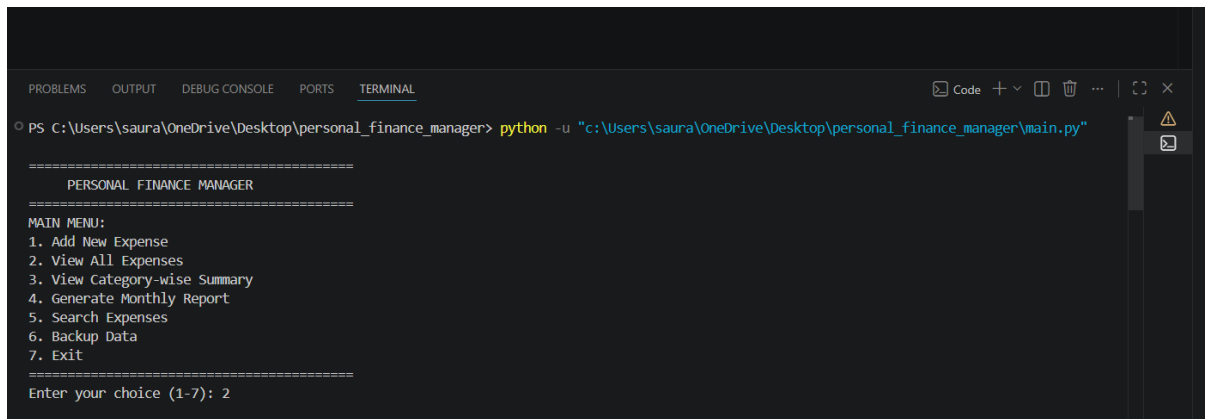
```
=====
PERSONAL FINANCE MANAGER
=====
```

MAIN MENU:

1. Add New Expense
2. View All Expenses
3. View Category-wise Summary
4. Generate Monthly Report
5. Search Expenses
6. Backup Data
7. Exit

```
=====
```

Enter your choice (1-7):



```
PROBLEMS OUTPUT DEBUG CONSOLE PORTS TERMINAL
PS C:\Users\saura\OneDrive\Desktop\personal_finance_manager> python -u "c:\Users\saura\OneDrive\Desktop\personal_finance_manager\main.py"

=====
PERSONAL FINANCE MANAGER
=====
MAIN MENU:
1. Add New Expense
2. View All Expenses
3. View Category-wise Summary
4. Generate Monthly Report
5. Search Expenses
6. Backup Data
7. Exit
=====
Enter your choice (1-7): 2
```

Step-by-Step Option Walkthroughs

Option 1: Add New Expense

Allows you to record a new transaction with real-time validation checks.

- **Execution steps:**

1. Press 1 and hit Enter.
2. Input a positive numerical value. If you type letters or negative numbers, the system displays an error and prompts you again.
3. Enter one of the whitelisted categories: Food, Transport, Entertainment, Shopping, or Other.
4. Enter a date (YYYY-MM-DD) or press Enter to automatically use the current system date.
5. Enter a brief description.

- **Console Dialog Example:**

codeText

ADD NEW EXPENSE:

Enter amount: 1000

Enter category (Food/Transport/Entertainment/Shopping/Other): Food

Enter date (YYYY-MM-DD) or press Enter for today: 2026-08-10

Enter description: Weekly groceries

✓ Expense added successfully!

```
PROBLEMS OUTPUT DEBUG CONSOLE PORTS TERMINAL
PS C:\Users\saura\OneDrive\Desktop\personal_finance_manager> python -u "c:\Users\saura\OneDrive\Desktop\personal_finance_manager\main.py"

Press Enter to continue...

=====
PERSONAL FINANCE MANAGER
=====
MAIN MENU:
1. Add New Expense
2. View All Expenses
3. View Category-wise Summary
4. Generate Monthly Report
5. Search Expenses
6. Backup Data
7. Exit
=====
Enter your choice (1-7): 1

ADD NEW EXPENSE:
Enter amount: 750
Enter category (Food/Transport/Entertainment/Shopping/Other): entertainment
Enter date (YYYY-MM-DD) or press Enter for today: 2026-08-11
Enter description: movie
```

Option 2: View All Expenses

Displays all recorded transactions in a formatted table.

- **Console Dialog Example:**

codeText

ALL EXPENSES:

Date	Category	Amount	Description

2026-08-10	Food	₹1000.00	Weekly groceries
2026-08-10	Transport	₹150.00	Bus ride to work

```
=====
MAIN MENU:
1. Add New Expense
2. View All Expenses
3. View Category-wise Summary
4. Generate Monthly Report
5. Search Expenses
6. Backup Data
7. Exit
=====
Enter your choice (1-7): 2

ALL EXPENSES:
-----
Date      | Category      | Amount    | Description
-----
2026-08-10 | Food          | ₹1000.00  | Weekly groceries
2026-08-11 | Entertainment | ₹750.00   | movie
2026-08-12 | Other         | ₹10000.00 | Room rent
2026-08-14 | Other         | ₹2000.00  | No description provided
2026-08-20 | Other         | ₹10000.00 | festival
2026-08-27 | Shopping      | ₹5000.00  | cloths,etc.
-----
Press Enter to continue...|
```

Option 3: View Category-wise Summary

Displays cumulative totals grouped by categories.

- **Console Dialog Example:**

codeText

CATEGORY-WISE SUMMARY:

Category	Total Spent

Food	₹1000.00
Transport	₹150.00

TOTAL	₹1150.00

```
PERSONAL FINANCE MANAGER
=====
MAIN MENU:
1. Add New Expense
2. View All Expenses
3. View Category-wise Summary
4. Generate Monthly Report
5. Search Expenses
6. Backup Data
7. Exit
=====
Enter your choice (1-7): 3

CATEGORY-WISE SUMMARY:
-----
Category          | Total Spent
-----
Food               | ₹1000.00
Entertainment      | ₹750.00
Other              | ₹22000.00
Shopping           | ₹5000.00
-----
TOTAL              | ₹28750.00
-----
Press Enter to continue...|
```

Option 4: Generate Monthly Report

Provides a detailed breakdown of a given month, with options to export the report.

- **Console Dialog Example:**

codeText

GENERATE MONTHLY REPORT:

Enter Month (YYYY-MM): 2026-08

```
=====
REPORT FOR 2026-08
=====
```

Total Transactions: 2

Total Expenditure: ₹1150.00

Average Expense: ₹575.00

Category Breakdown:

- Food : ₹1000.00 (87.0%)
 - Transport : ₹150.00 (13.0%)
- =====

Would you like to export this report to a text file? (y/n): y

☒ Report exported successfully to:

reports\report_2026-08.txt

```
4. Generate Monthly Report
5. Search Expenses
6. Backup Data
7. Exit
```

```
=====
Enter your choice (1-7): 4
```

```
GENERATE MONTHLY REPORT:
Enter Month (YYYY-MM): 2026-08
```

```
=====
REPORT FOR 2026-08
=====
```

```
Total Transactions: 6
Total Expenditure: ₹28750.00
Average Expense: ₹4791.67
```

```
-----
Category Breakdown:
```

- Food : ₹1000.00 (3.5%)
 - Entertainment: ₹750.00 (2.6%)
 - Other : ₹22000.00 (76.5%)
 - Shopping : ₹5000.00 (17.4%)
- ```
=====
```

```
Press Enter to continue...
```

---

## Option 5: Search Expenses

Allows you to query recorded transactions using search terms.

- **Console Dialog Example:**

codeText

SEARCH EXPENSES:

Enter search keyword (category or description): grocer

Found 1 matches:

-----  
2026-08-10 | Food: ₹1000.00 - Weekly groceries  
-----

```
=====
 PERSONAL FINANCE MANAGER
=====
MAIN MENU:
1. Add New Expense
2. View All Expenses
3. View Category-wise Summary
4. Generate Monthly Report
5. Search Expenses
6. Backup Data
7. Exit
=====
Enter your choice (1-7): 6

BACKING UP DATA...
✅ Backup created successfully at:
 data\backups\expenses_backup_20260831_192220.csv

Press Enter to continue...

=====
 PERSONAL FINANCE MANAGER
=====
```

---

## Option 6: Backup Data

Generates a dated backup copy of your active ledger file.

- **Console Dialog Example:**

codeText



BACKING UP DATA...

✅ Backup created successfully at:  
data\backups\expenses\_backup\_20260810\_133000.csv

---

### Option 7: Exit

Safely closes the application.

- **Console Dialog Example:**

codeText

Exiting program. Goodbye!

```
BACKING UP DATA...
✅ Backup created successfully at:
 data\backups\expenses_backup_20260831_192220.csv

Press Enter to continue...

=====
 PERSONAL FINANCE MANAGER
=====
MAIN MENU:
1. Add New Expense
2. View All Expenses
3. View Category-wise Summary
4. Generate Monthly Report
5. Search Expenses
6. Backup Data
7. Exit
=====
Enter your choice (1-7): 7

Exiting program. Goodbye!
PS C:\Users\saura\OneDrive\Desktop\personal_finance_manager> |
```

---

## 7. Visual Documentation Plan

To satisfy the **Visual Documentation** section of the submission checklist, take screenshots of your terminal running the application and save them in a folder named screenshots/.

Refer to this plan for the required images:

| <b>Screenshot ID</b> | <b>Recommended File Name</b> | <b>Console View Content to Capture</b>                                  |
|----------------------|------------------------------|-------------------------------------------------------------------------|
| <b>Screenshot 1</b>  | 01_main_menu.png             | Run python main.py and capture the main menu interface.                 |
| <b>Screenshot 2</b>  | 02_add_expense.png           | Run Option 1 and capture the successful entry of an expense.            |
| <b>Screenshot 3</b>  | 03_all_expenses.png          | Run Option 2 and capture the formatted transaction table.               |
| <b>Screenshot 4</b>  | 04_monthly_report.png        | Run Option 4 and capture the monthly calculations and export message.   |
| <b>Screenshot 5</b>  | 05_backup.png                | Run Option 6 and capture the backup file generation confirmation path.  |
| <b>Screenshot 6</b>  | 06_unit_tests.png            | Run the test command and capture the output showing Ran 4 tests ... OK. |

---

## 8. Testing Evidence

Automated validation testing is performed using Python's built-in unittest module inside tests/test\_finance.py.

### Test Cases Explained

1. **test\_validate\_amount:** Tests validation rules for amounts. Confirms that decimal formats (e.g., 100.50) are accepted, and letters (abc), zeroes (0), and negative values (-50) are caught and rejected.
2. **test\_validate\_date:** Tests validation rules for dates. Confirms that standard structures like YYYY-MM-DD pass, and malformed layouts like DD-MM-YYYY are caught and rejected.
3. **test\_validate\_category:** Tests that categories are standardized correctly. Confirms that spaces and variations in capitalization (e.g., food) are normalized to Food, and non-whitelisted categories are caught and rejected.
4. **test\_category\_summary:** Verifies that mathematics and grouping logic calculates accurate totals across distinct categories.

### Command to Execute Test Suites

Run the following command in the project root directory:

codeBash

```
python -m unittest tests/test_finance.py
```

### Test Output Log (Evidence Proof)

codeText

```
PS C:\Users\saura\OneDrive\Desktop\personal_finance_manager> python -m
unittest tests/test_finance.py
```

```
....
```

```

```

```
Ran 4 tests in 0.005s
```

```
OK
```

---

## 9. Troubleshooting Guide

This guide addresses common errors and issues you may encounter when setting up or running the application.

### Scenario A: `ModuleNotFoundError: No module named 'src'`

- **Error Message:**

codeText

Traceback (most recent call last):

File "src/file\_manager.py", line 5, in <module>

from src.expense import Expense

ModuleNotFoundError: No module named 'src'

- **Root Cause:** Attempting to execute modular script files inside the `/src/` subfolder directly. The imports inside the application are relative to the project root directory [1].
- **Resolution:** Always launch operations from the parent root folder (`personal_finance_manager/`) [1]. Use `python main.py` or `python -m unittest tests/test_finance.py`. Do not run files from inside `/src` directly.

---

### Scenario B: Asterisk [\*] displayed on screen or terminal freeze

- **Error Message:** The compiler runs indefinitely and does not respond to input.
  - **Root Cause:** Attempting to run interactive console loops (like `input()`) inside a notebook interface (such as Jupyter Notebook). These loops can block the notebook kernel.
  - **Resolution:** Close the notebook runtime and open your standard terminal shell (PowerShell or Bash). Run the application using standard Python commands.
-

## Scenario C: Permission Denied Writing to Data Files

- **Error Message:**

codeText

PermissionError: [Errno 13] Permission denied: 'data/expenses.csv'

- **Root Cause:** The active CSV file is open in an external spreadsheet program (such as Excel), which locks the file and prevents Python from writing to it.
  - **Resolution:** Close the CSV file in Excel or other external applications and run the terminal command again.
- 

## Scenario D: FileNotFoundError on startup

- **Error Message:** The application fails because it cannot find the data folder or expenses.csv file.
- **Root Cause:** The directory structure is empty when the project is first created or cloned.
- **Resolution:** No manual action is required. The application's `ensure_data_dir()` helper checks and automatically generates any missing files or folders on startup [1]. Simply run `main.py` once to generate these directories.